МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное бюджетное образовательное учреждение высшего образования

«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Сопроводительная записка

по производственной практике

ПП.02.01 Технология разработки программного обеспечения

(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Марфина Анастасия Станиславовна

(подпись) (фамилия, имя, отчество)

Группы ИПо-42

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович

(подпись) (фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

Содержание

Раздел 1. MarfinaWpfApp03 3
1.1. ViewModels 4
1.2. Главное окно 7
1.3. Окно редактирования партнёров 13
1.4. Окно редактирования продаж 18
Раздел 2. MarfinaLibrary 24
2.1. Диаграмма классов 26
2.2. Database 28
2.3. Models 30
2.4. Repositories 35
2.5. Services 39
Раздел 3. Схема базы данных 45
Раздел 4. Тестирование 48

Раздел 1. MarfinaWpfApp03

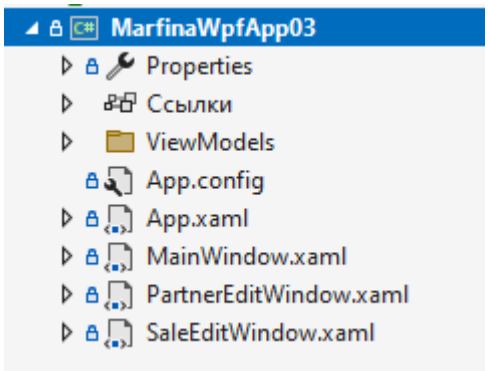

Рисунок 1 – Структура MarfinaWpfApp03

Проект MarfinaWpfApp03 представляет собой клиентское приложение, реализующее пользовательский интерфейс для работы с информацией о партнерах и их продажах. Проект разработан с использованием технологий Windows Presentation Foundation (WPF) и следует паттерну MVVM (Model-View-ViewModel), что обеспечивает четкое разделение логики представления от бизнес-логики и данных.

Структура проекта организована следующим образом. Корневая директория содержит файлы конфигурации и определения приложения. Файл App.config предназначен для хранения параметров конфигурации, таких как строки подключения к базе данных и настройки среды выполнения.

Директория ViewModels содержит класс PartnerViewModel.cs, реализующий модель представления для главного окна. Данный класс выступает посредником между пользовательским интерфейсом и уровнем доступа к данным, предоставляя коллекции для отображения, команды для обработки действий пользователя и реализует интерфейс для автоматического обновления связанных элементов интерфейса при изменении данных.

Главное окно приложения представлено файлами MainWindow.xaml и MainWindow.xaml.cs. Файл разметки определяет структуру пользовательского интерфейса, включая меню, список партнеров с отображением персональной информации и рассчитанной скидки, а также таблицу истории продаж для выбранного партнера. Файл кода программной части содержит обработчики событий для элементов управления, методы для открытия диалоговых окон и формирования отчетов.

Для выполнения операций добавления и редактирования данных о партнерах в проекте реализовано диалоговое окно, представленное файлами PartnerEditWindow.xaml и PartnerEditWindow.xaml.cs. Файл разметки содержит форму с полями для ввода информации о партнере, включая выбор типа из выпадающего списка, ввод наименования, рейтинга, контактных данных и адреса. Файл кода программной части реализует логику загрузки данных при редактировании, валидацию вводимых значений и сохранение информации через соответствующий репозиторий.

1.1. ViewModels

1.1.1. PartnerViewModel

Для обеспечения взаимодействия между пользовательским интерфейсом и бизнес-логикой приложения разработан класс PartnerViewModel, реализующий паттерн Model-View-ViewModel (MVVM). Класс выступает в роли посредника, предоставляя данные для отображения и обрабатывая команды пользователя, а также реализует интерфейс INotifyPropertyChanged для автоматического обновления элементов интерфейса при изменении данных.

В конструкторе ViewModel создаются экземпляры всех необходимых репозиториев и сервисов: PartnerRepository для доступа к данным партнеров, SalesRepository для получения информации о продажах, DiscountService для расчета скидок и SalesService для подготовки данных о продажах к отображению. Инициализируются наблюдаемые коллекции Partners и SalesHistory, после чего асинхронно запускается загрузка списка партнеров:

```
public PartnerViewModel()
{
    _partnerRepository = new PartnerRepository();
    _salesRepository = new SalesRepository();
    _discountService = new DiscountService();
    _salesService = new SalesService();
    Partners = new ObservableCollection<PartnerDisplayModel>();
    SalesHistory = new ObservableCollection<SalesDisplayModel>();
    _ = LoadPartnersAsync();
}
```

Ключевыми свойствами ViewModel являются наблюдаемые коллекции и выбранные элементы. При изменении выбранного партнера автоматически запускается загрузка истории его продаж благодаря вызову в сеттере свойства SelectedPartner. Все свойства уведомляют об изменениях через метод OnPropertyChanged, что обеспечивает синхронизацию с пользовательским интерфейсом:

```
public PartnerDisplayModel SelectedPartner
{
    get => _selectedPartner;
    set
    {
        _selectedPartner = value;
        OnPropertyChanged();
        if (value != null)
        {
            _ = LoadSalesHistoryAsync(value.Id);
        }
    }
}
```

Метод LoadPartnersAsync выполняет загрузку списка партнеров из репозитория, для каждого партнера запрашивает общую сумму продаж и рассчитывает скидку с использованием DiscountService. Полученные данные преобразуются в модель отображения PartnerDisplayModel и добавляются в наблюдаемую коллекцию. В процессе загрузки управляется состояние индикатора загрузки через свойство IsLoading:

```
foreach (var partner in partners)
{
    var totalSales = await _salesRepository.GetTotalSalesByPartnerAsync(partner.Id);
    var discountPercent = _discountService.CalculateDiscount(totalSales);

    Partners.Add(new PartnerDisplayModel
```

```
        {
            Id = partner.Id,
            Name = partner.Name,
            TypeName = partner.PartnerType?.Name ?? "Не указан",
            DiscountPercent = discountPercent,
            DiscountDescription = _discountService.GetDiscountDescription(discountPercent)
        });
    }
```

Для выполнения операций добавления, обновления и удаления партнеров реализованы соответствующие методы, которые взаимодействуют с репозиторием и обновляют отображение после успешного выполнения. Методы содержат подтверждение операций через диалоговые окна и обрабатывают возможные исключения с выводом информативных сообщений пользователю. Пример метода удаления партнера:

```
    public async Task<bool> DeletePartnerAsync(int id)
    {
        var result = MessageBox.Show("Вы уверены, что хотите удалить партнера?",
            "Подтверждение удаления", MessageBoxButton.YesNo,
MessageBoxImage.Question);

        if (result == MessageBoxResult.Yes)
        {
            await _partnerRepository.DeletePartnerAsync(id);
            await RefreshPartnersAsync();
            return true;
        }
        return false;
    }
```

Для представления данных о партнере в пользовательском интерфейсе разработана модель PartnerDisplayModel, которая содержит только те поля, которые необходимы для отображения, а также вычисляемое свойство DisplayName для удобного форматирования:

```
    public class PartnerDisplayModel
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string TypeName { get; set; }
        public int DiscountPercent { get; set; }
        public string DiscountDescription { get; set; }
        public string DisplayName => $"{Name} (скидка: {DiscountDescription})";
    }
```

Реализация PartnerViewModel обеспечивает полную функциональность для работы с данными партнеров на уровне представления, включает

механизмы загрузки, обновления и модификации данных, управляет состоянием пользовательского интерфейса и обрабатывает пользовательские действия, что соответствует требованиям паттерна MVVM и обеспечивает четкое разделение логики приложения и его представления.

1.2. Главное окно

1.2.1. MainWindow

Для обеспечения взаимодействия пользователя с системой разработано главное окно приложения MainWindow, реализованное с использованием декларативного языка разметки XAML. Интерфейс окна построен на основе паттерна MVVM и обеспечивает отображение списка партнеров, истории продаж, а также предоставляет навигацию по функциям системы через меню.

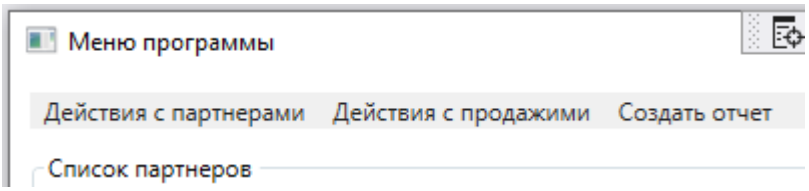

Рисунок 2 - Меню программы

В ресурсах окна определены стили для основных элементов интерфейса, обеспечивающие единообразное визуальное оформление. Стиль ButtonStyle задает внешний вид кнопок с синим фоном и эффектом при наведении, а DangerButtonStyle наследует базовый стиль и изменяет цвет фона на красный для обозначения опасных действий. Также определен конвертер BooleanToVisibilityConverter для управления видимостью элементов на основе булевых значений:

```
<BooleanToVisibilityConverter x:Key="BoolToVisConverter"/>
```

Верхняя часть окна содержит меню с пунктами для выполнения основных операций. Пункты редактирования и удаления партнера привязаны к свойству SelectedPartner через конвертер, что обеспечивает их доступность только при наличии выбранного элемента в списке:

```
<MenuItem Header="Редактировать партнера"
Click="EditPartner_Click"
IsEnabled="{Binding SelectedPartner, Converter={StaticResource
BoolToVisConverter}}"/>
```

Основная область окна разделена на две колонки с помощью элемента Grid. Левая колонка фиксированной ширины содержит список партнеров, правая занимает оставшееся пространство и отображает историю продаж выбранного партнера. Между колонками размещен разделитель GridSplitter, позволяющий пользователю изменять ширину колонок в процессе работы.

Список партнеров реализован с использованием элемента ListBox, источником данных которого служит коллекция Partners из ViewModel. Для каждого элемента списка определен шаблон DataTemplate, формирующий карточку партнера с отображением типа, наименования, контактной информации и рассчитанного процента скидки:

```
<TextBlock Grid.Column="1"
  HorizontalAlignment="Right"
  VerticalAlignment="Center"
  FontWeight="Bold"
  FontSize="16"
  Text="{Binding DiscountPercent, StringFormat={}{0}%}" />
```

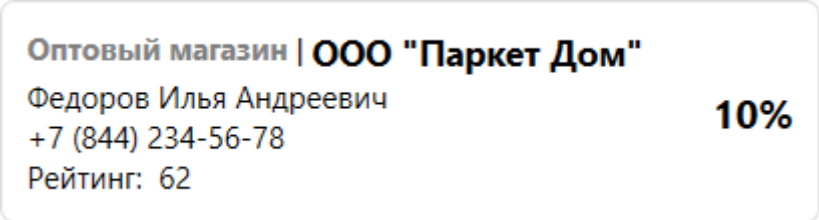

Рисунок 3 – Отображение партнеров

При выборе партнера в списке его данные отображаются в информационном блоке правой колонки, видимость которого управляется привязкой к выбранному элементу. История продаж представляется в элементе DataGrid с предопределенными колонками для наименования продукции, количества, даты продажи и суммы. Форматирование даты и суммы выполняется с использованием строк формата:

```
<DataGridTextColumn Header="Дата продажи"
        Binding="{Binding SaleDate, StringFormat=dd.MM.yyyy}"
        Width="100"/>
```

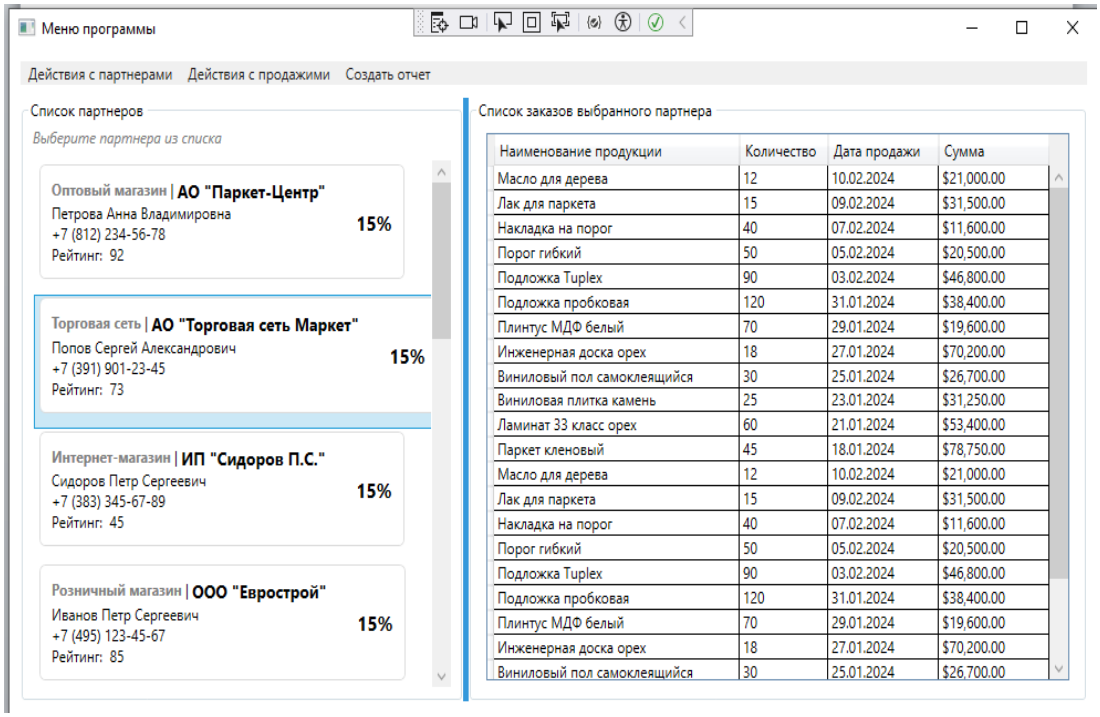

Рисунок 4 – Главное окно

Реализация главного окна обеспечивает интуитивно понятный интерфейс для работы с системой, включает необходимые элементы навигации и управления, а также полностью интегрирована с ViewModel через механизм привязки данных, что гарантирует автоматическое обновление интерфейса при изменении состояния модели представления.

1.2.2. MainWindow.cs

Для обеспечения функциональности главного окна приложения разработан класс MainWindow, реализующий логику обработки пользовательских действий и взаимодействия с ViewModel. Класс содержит обработчики событий для элементов меню и управления данными, а также методы для формирования отчетов и обновления информации.

В конструкторе окна выполняется инициализация компонентов, создается экземпляр PartnerViewModel, который устанавливается в качестве контекста данных. Это обеспечивает связывание пользовательского интерфейса с данными и командами, определенными в ViewModel:

```
public MainWindow()
{
    InitializeComponent();
    _viewModel = new PartnerViewModel();
```

```
    DataContext = _viewModel;
}
```

Обработчики добавления и редактирования партнера реализованы с использованием модальных окон PartnerEditWindow. При успешном закрытии окна с результатом true выполняется асинхронное обновление списка партнеров через вызов метода RefreshPartnersAsync без ожидания завершения, что обеспечивает немедленное закрытие диалога и фоновое обновление данных. Пример обработчика добавления:

```
private void AddPartner_Click(object sender, RoutedEventArgs e)
{
    var editWindow = new PartnerEditWindow();
    editWindow.Owner = this;

    if (editWindow.ShowDialog() == true)
    {
        _ = _viewModel.RefreshPartnersAsync();
    }
}
```

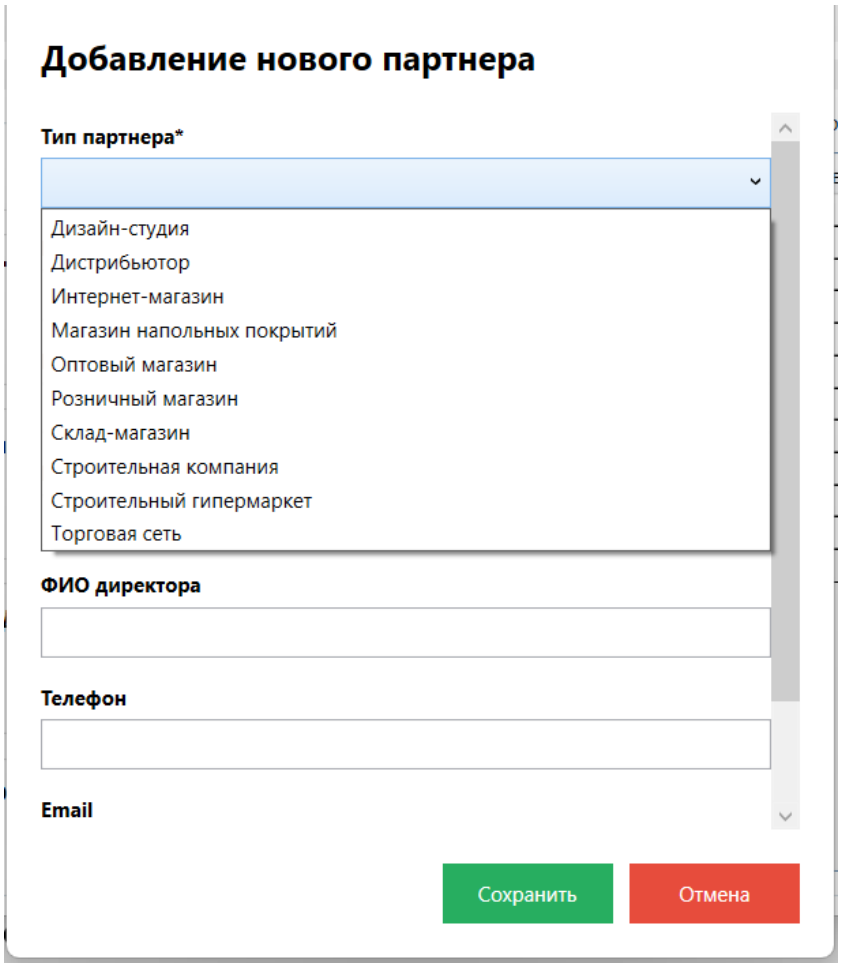

Рисунок 5 – Добавление нового партнера

Обработчик удаления партнера реализован с асинхронным паттерном async void, так как он обрабатывает событие и должен поддерживать ожидание завершения операции. Перед выполнением удаления выполняется проверка наличия выбранного партнера, после чего вызывается соответствующий метод ViewModel:

```
private async void DeletePartner_Click(object sender, RoutedEventArgs e)
{
    if (_viewModel.SelectedPartner == null)
    {
        MessageBox.Show("Выберите партнера для удаления", "Предупреждение",
            MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    await _viewModel.DeletePartnerAsync(_viewModel.SelectedPartner.Id);
}
```

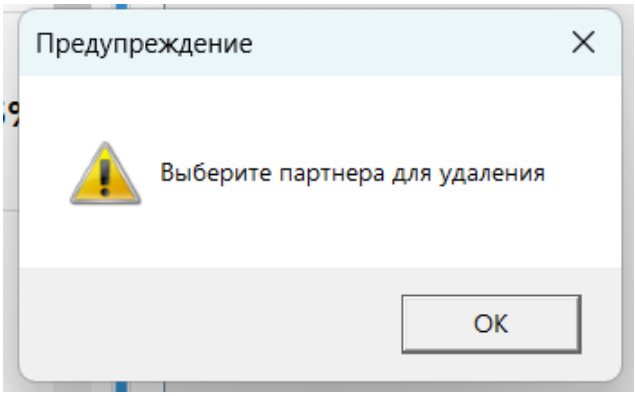

Рисунок 6 – Предупреждение об выборе партнера

Для обработки редактирования записей в таблице продаж реализован метод SalesGrid_RowEditEnding, который срабатывает при завершении редактирования строки в DataGrid. Метод извлекает измененную запись, создает объект SalesHistory и обновляет его через репозиторий продаж:

```
private async void SalesGrid_RowEditEnding(object sender,
DataGridRowEditEndingEventArgs e)
{
    if (e.Row.Item is SalesDisplayModel row)
    {
        var sale = new SalesHistory
        {
            PartnerId = _viewModel.SelectedPartner.Id,
            Quantity = row.Quantity,
            SaleDate = row.SaleDate,
            TotalAmount = row.TotalAmount
        };

        await _salesRepository.UpdateSaleAsync(sale);
```

```
    }
}
```

Функциональность формирования отчета реализована методом GenerateReport, который формирует текстовый отчет на основе данных из коллекции партнеров ViewModel. Отчет включает наименование, тип, рейтинг, контактную информацию и данные о директоре каждого партнера. Метод CreateReport_Click сохраняет сгенерированный отчет в текстовый файл report.txt в директории приложения и открывает его для просмотра с помощью стандартного приложения "Блокнот":

```
private void CreateReport_Click(object sender, RoutedEventArgs e)
{
    var reportContent = GenerateReport();
    string filePath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "report.txt");
    File.WriteAllText(filePath, reportContent);
    System.Diagnostics.Process.Start("notepad.exe", filePath);
}
```

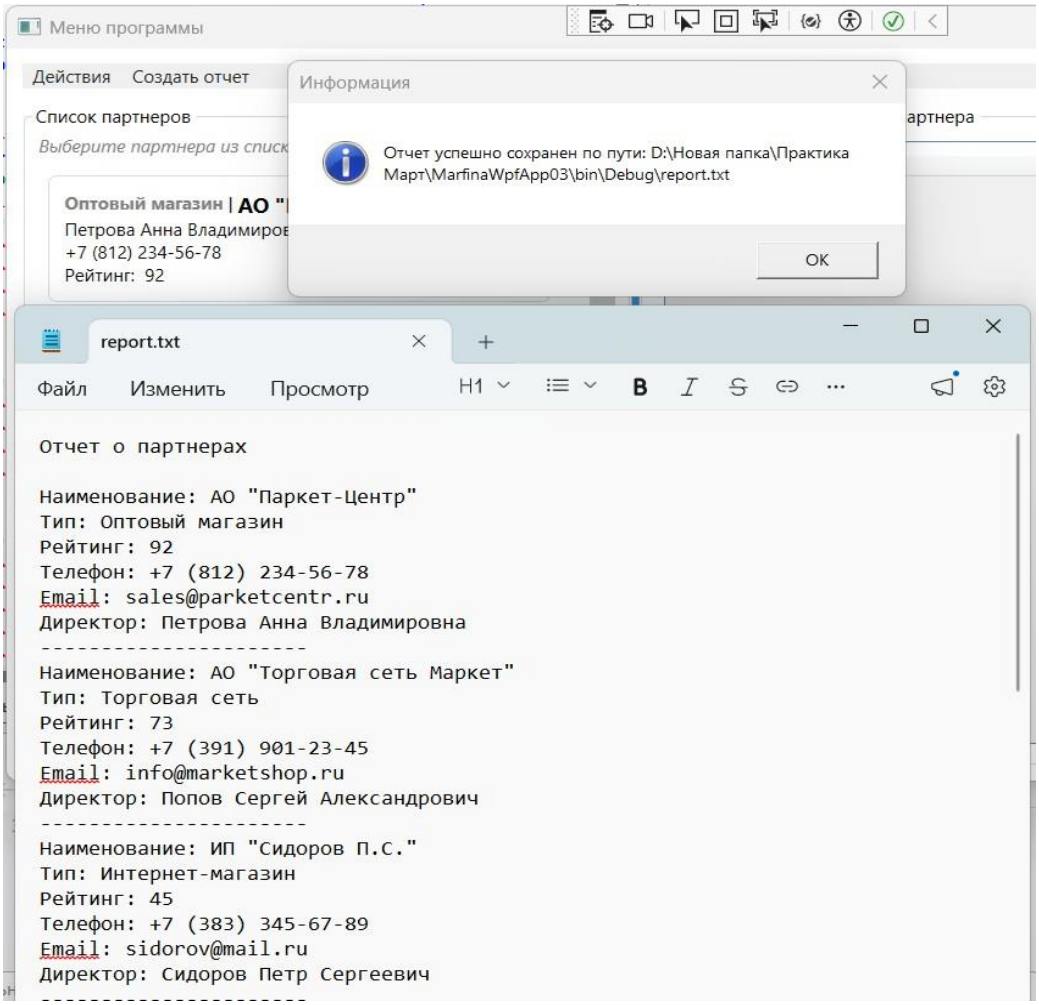

Рисунок 7 – Создание отчета

Все методы обработки событий содержат блоки обработки исключений с выводом информативных сообщений пользователю через диалоговые окна MessageBox. Это обеспечивает корректную обратную связь при возникновении ошибок и повышает надежность приложения.

Реализация программной части главного окна обеспечивает полную обработку пользовательских действий, взаимодействие с ViewModel и репозиториями, формирование отчетов и асинхронное обновление данных, что создает законченный пользовательский интерфейс для управления информацией о партнерах и их продажах.

1.3. Окно редактирования партнёров

1.3.1. PartnerEditWindow

Для обеспечения ввода и редактирования данных о партнерах разработано диалоговое окно PartnerEditWindow, реализованное с использованием декларативного языка разметки XAML. Окно предоставляет форму для заполнения информации о партнере с разделением на обязательные и опциональные поля, а также содержит элементы управления для сохранения или отмены изменений.

Окно имеет фиксированный размер 600 на 500 пикселей и позиционируется по центру относительно владельца (CenterOwner), что обеспечивает модальное поведение при вызове из главного окна. Запрет изменения размера (ResizeMode="NoResize") гарантирует стабильность макета формы:

```
Title="Редактирование партнера"
Height="600" Width="500"
WindowStartupLocation="CenterOwner"
ResizeMode="NoResize">
```

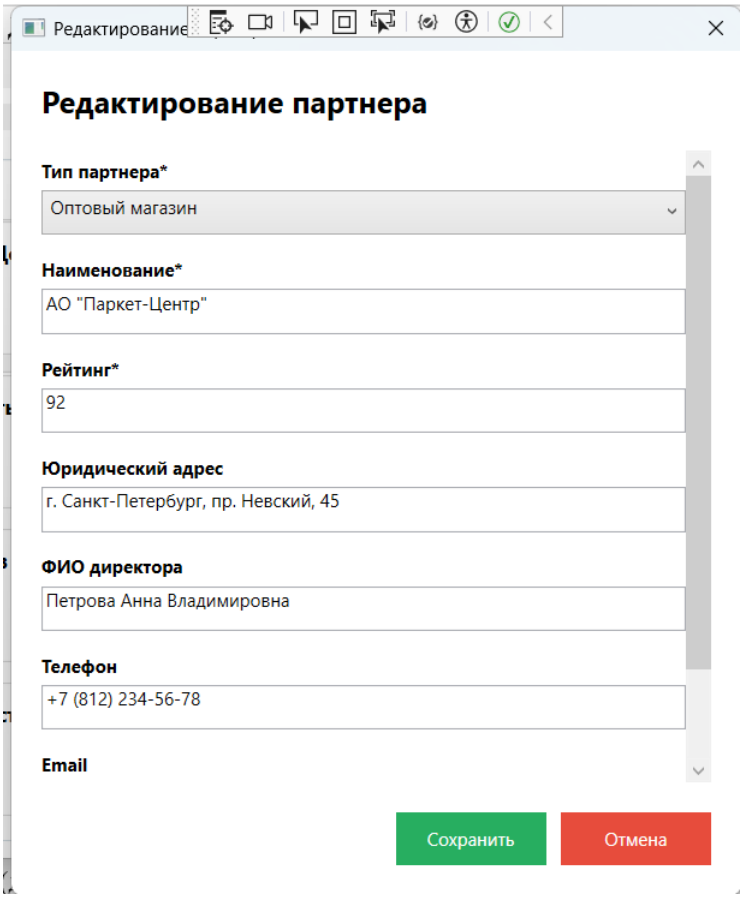

Рисунок 8 – Редактирование партнера

Структура окна организована с помощью элемента Grid, разделенного на три строки: область заголовка, область формы с прокруткой и область кнопок. Заголовок представлен элементом TextBlock с именем TitleTextBlock, что позволяет динамически изменять текст в зависимости от режима работы окна (добавление или редактирование).

Форма ввода размещена в ScrollViewer, что обеспечивает доступность всех полей при любом разрешении экрана. Поля формы сгруппированы в вертикальный StackPanel и включают:

- выпадающий список для выбора типа партнера с отображением наименования и хранением идентификатора;

- текстовые поля для ввода наименования, рейтинга, юридического адреса, ФИО директора, телефона и электронной почты.

Для поля рейтинга реализована предварительная валидация ввода с помощью обработчика PreviewTextInput, ограничивающего ввод только

цифровыми символами. Обязательность полей обозначена символом "*" в подписях и отдельным пояснением в нижней части формы:

```
<TextBlock Text="* - обязательные поля"
        Foreground="Gray"
        FontSize="11"
        Margin="0,10"/>
```

В нижней части окна расположены кнопки управления. Кнопка сохранения имеет зеленый фон, кнопка отмены — красный, что соответствует общепринятой цветовой схеме для подтверждения и отказа от действий. Обе кнопки обрабатывают события клика соответствующими методами в коде программной части:

```
<Button Content="Сохранить" Click="Save_Click" Background="#27ae60"/>
<Button Content="Отмена" Click="Cancel_Click" Background="#e74c3c"/>
```

Реализация окна редактирования партнера обеспечивает интуитивно понятный интерфейс для ввода данных, включает визуальные подсказки об обязательности полей, предотвращает некорректный ввод в числовых полях и предоставляет стандартные элементы управления для сохранения или отмены изменений, что соответствует требованиям к пользовательскому интерфейсу корпоративных приложений.

1.3.2. PartnerEditWindow.cs

Для обеспечения функциональности окна редактирования данных партнера разработан класс PartnerEditWindow, реализующий логику загрузки, валидации и сохранения информации о партнере. Класс поддерживает два режима работы: добавление нового партнера и редактирование существующего, что реализовано с помощью конструкторов.

В конструкторе для режима добавления выполняются инициализация компонентов, создание экземпляра репозитория и установка соответствующего заголовка окна. После этого асинхронно загружается список типов партнеров для заполнения выпадающего списка:

```
public PartnerEditWindow()
{
   InitializeComponent();
   _partnerRepository = new PartnerRepository();
```

```
    TitleTextBlock.Text = "Добавление нового партнера";
    LoadPartnerTypesAsync();
}
```

Конструктор для режима редактирования принимает идентификатор партнера, сохраняет его для последующего использования и инициирует последовательную загрузку данных через метод LoadDataAsync. Такой подход гарантирует корректный порядок загрузки: сначала типы партнеров, затем данные конкретного партнера:

```
private async void LoadDataAsync()
{
    await LoadPartnerTypesTaskAsync();
    await LoadPartnerDataAsync();
}
```

Метод LoadPartnerDataAsync выполняет получение данных партнера из репозитория и заполняет поля формы. При этом используется оператор условного доступа для защиты от null-значений, а выбранное значение в выпадающем списке устанавливается через SelectedValue:

```
TypeComboBox.SelectedValue = _currentPartner.TypeId;
NameTextBox.Text = _currentPartner.Name;
RatingTextBox.Text = _currentPartner.Rating.ToString();
```

Для обеспечения отказоустойчивости в классе предусмотрены альтернативные методы параллельной загрузки данных с использованием отдельных экземпляров репозитория и маршалинга обращений к пользовательскому интерфейсу через Dispatcher:

```
Dispatcher.Invoke(() => {
    TypeComboBox.ItemsSource = types;
});
```

Метод сохранения Save_Click реализует комплексную логику: валидацию входных данных, формирование объекта партнера, выбор между операциями добавления и обновления, а также управление результатом диалога. Перед сохранением выполняется очистка строковых полей от пробелов и замена пустых значений на null:

```
partner.LegalAddress = string.IsNullOrWhiteSpace(AddressTextBox.Text) ? null : AddressTextBox.Text.Trim();
```

Валидация данных реализована в методе ValidateInput, который проверяет обязательность выбора типа и заполнения наименования, корректность рейтинга (целое неотрицательное число) и при наличии email — его соответствие стандартному формату. Для проверки email используется создание экземпляра MailAddress, что обеспечивает надежную валидацию без сложных регулярных выражений:

```
private bool IsValidEmail(string email)
{
    try
    {
        var addr = new System.Net.Mail.MailAddress(email);
        return addr.Address == email;
    }
    catch
    {
        return false;
    }
}
```

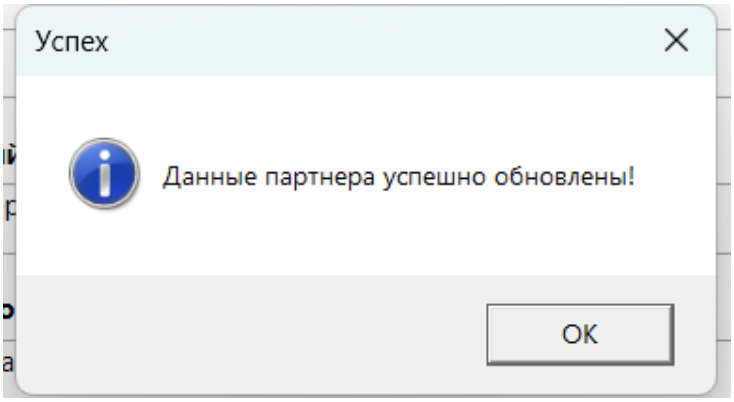

Рисунок 9 – Уведомление об успешном обновлении данных

Для поля ввода рейтинга реализована предварительная фильтрация ввода с помощью обработчика NumberValidationTextBox, который использует регулярное выражение для запрета ввода любых символов, кроме цифр. Это повышает удобство использования и предотвращает некорректный ввод на раннем этапе:

```
Regex regex = new Regex("[^0-9]+");
e.Handled = regex.IsMatch(e.Text);
```

Обработчик отмены Cancel_Click устанавливает отрицательный результат диалога и закрывает окно, что соответствует стандартному поведению модальных окон.

Все асинхронные методы содержат обработку исключений с выводом информативных сообщений пользователю через MessageBox, что обеспечивает обратную связь при возникновении ошибок загрузки или сохранения данных.

Реализация программной части окна редактирования партнера обеспечивает полный цикл работы с данными: загрузку справочной информации, отображение существующих данных, валидацию пользовательского ввода и сохранение изменений, что создает надежный и удобный интерфейс для управления информацией о партнерах в системе.

1.4. Окно редактирования продаж

1.4.1. SaleEditWindow

Для обеспечения ввода и редактирования данных о продажах разработано диалоговое окно `SaleEditWindow`, реализованное с использованием декларативного языка разметки XAML. Окно предоставляет форму для заполнения информации о продаже с автоматическим расчетом суммы на основе выбранного продукта и указанного количества, а также содержит элементы управления для сохранения, отмены или удаления записи.

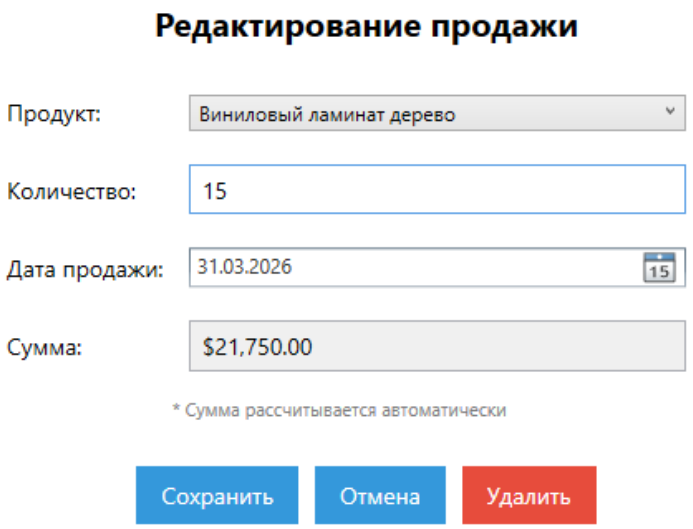

Рисунок 10 – Редактирование продаж

Окно имеет фиксированный размер 500 на 400 пикселей и позиционируется по центру относительно владельца (`CenterOwner`), что обеспечивает модальное поведение при вызове из главного окна. Запрет

изменения размера (`ResizeMode="NoResize"`) гарантирует стабильность макета формы:

Title="Редактирование продажи" Height="400" Width="500" WindowStartupLocation="CenterOwner" ResizeMode="NoResize"

Структура окна организована с помощью элемента `Grid`, разделенного на семь строк: область динамического заголовка, поля выбора продукта, ввода количества, выбора даты, отображения суммы, информационная строка и область кнопок. Заголовок представлен элементом `TextBlock` с привязкой к свойству `WindowTitle`, что позволяет динамически изменять текст в зависимости от режима работы окна (добавление или редактирование).

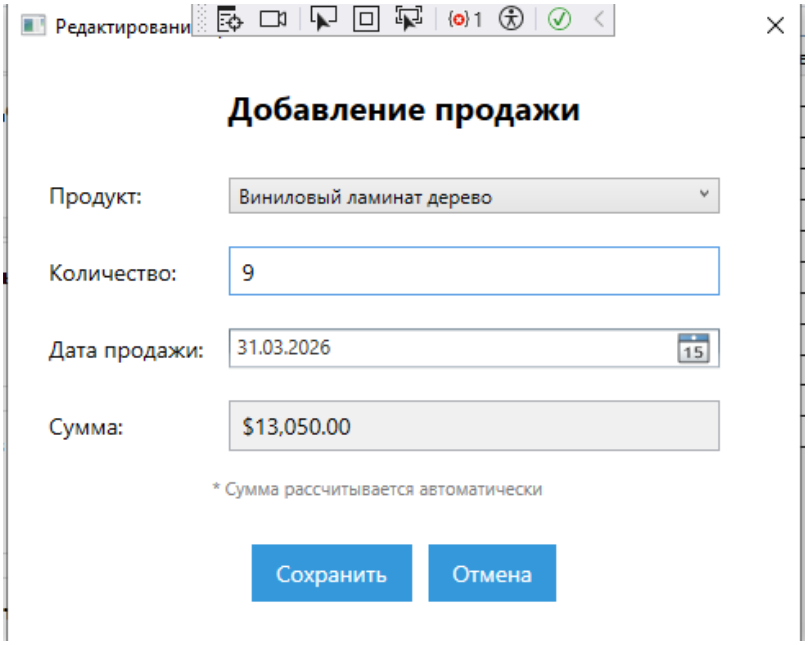

Рисунок 11 – Добавление продажи

Форма ввода включает следующие элементы:

- выпадающий список (`ComboBox`) для выбора продукта с отображением наименования и хранением идентификатора;
- текстовое поле (`TextBox`) для ввода количества продажи;
- элемент управления `DatePicker` для выбора даты продажи;
- текстовое поле для отображения суммы продажи, доступное только для чтения (`IsReadOnly="True"`), с фоном серого цвета для визуального выделения.

Для поля количества на уровне программной логики реализована валидация, проверяющая ввод положительного числа. Поле суммы обновляется

автоматически при изменении выбранного продукта или количества, используя значение минимальной цены для партнера из справочника продуктов. Обязательность полей обозначена символом "*" в информационной строке в нижней части формы:

```
<TextBlock Text="* Сумма рассчитывается автоматически" FontSize="11" Foreground="Gray"/>
```

В нижней части окна расположены кнопки управления. Кнопка сохранения имеет синий фон с эффектом наведения при наведении курсора, кнопка отмены стилизована аналогично, а кнопка удаления имеет красный фон и отображается только в режиме редактирования. Все кнопки обрабатывают события клика соответствующими методами в коде программной части:

```
<Button Content="Сохранить" Style="{StaticResource ButtonStyle}" Click="SaveButton_Click"/>
<Button Content="Отмена" Style="{StaticResource ButtonStyle}" Click="CancelButton_Click"/>
<Button Content="Удалить" Style="{StaticResource DangerButtonStyle}" Click="DeleteButton_Click" Visibility="Collapsed"/>
```

Реализация окна редактирования продаж обеспечивает интуитивно понятный интерфейс для ввода данных, включает автоматический расчет суммы на основе выбранного продукта и количества, предотвращает сохранение некорректных данных с выводом соответствующих предупреждений и предоставляет стандартные элементы управления для сохранения, отмены или удаления изменений, что соответствует требованиям к пользовательскому интерфейсу корпоративных приложений.

1.4.2. SaleEditWindow.cs

Для обеспечения функциональности окна редактирования данных о продаже разработан класс `SaleEditWindow`, реализующий логику загрузки, валидации, автоматического расчета суммы и сохранения информации. Класс поддерживает два режима работы: добавление новой продажи и редактирование существующей, что реализовано с помощью перегрузки конструкторов.

В конструкторе для режима добавления выполняется инициализация компонентов, создание экземпляров репозиториев для работы с продажами и продуктами, сохранение идентификатора партнера, установка контекста

данных и подписка на событие загрузки окна:

```
public SaleEditWindow(int partnerId) : this(partnerId, null) { }
```

Конструктор для режима редактирования принимает дополнительный идентификатор продажи, сохраняет его для последующего использования. Оба конструктора устанавливают `DataContext = this`, что обеспечивает привязку данных к элементам управления через механизм `INotifyPropertyChanged`.

Метод `LoadData` выполняется асинхронно при загрузке окна и обеспечивает последовательную загрузку справочника продуктов и данных конкретной продажи при редактировании. При этом используются репозитории `ProductRepository` и `SalesRepository` для получения данных. Для режима редактирования устанавливается соответствующий заголовок окна, отображается кнопка удаления, а поля формы заполняются полученными значениями:

```
if (_saleId.HasValue)
{
    WindowTitle = "Редактирование продажи";
    DeleteButton.Visibility = Visibility.Visible;
    var sale = await _salesRepository.GetSaleByIdAsync(_saleId.Value);
    if (sale != null)
    {
        ProductId = sale.ProductId;
        Quantity = sale.Quantity;
        SaleDate = sale.SaleDate;
        TotalAmount = sale.TotalAmount;
        ProductComboBox.SelectedValue = sale.ProductId;
    }
}
```

Для обеспечения отказоустойчивости все операции с данными обернуты в блоки `try-catch` с выводом информативных сообщений пользователю через `MessageBox`, что подтверждается скриншотами интерфейса с сообщениями об ошибках и успешном выполнении операций.

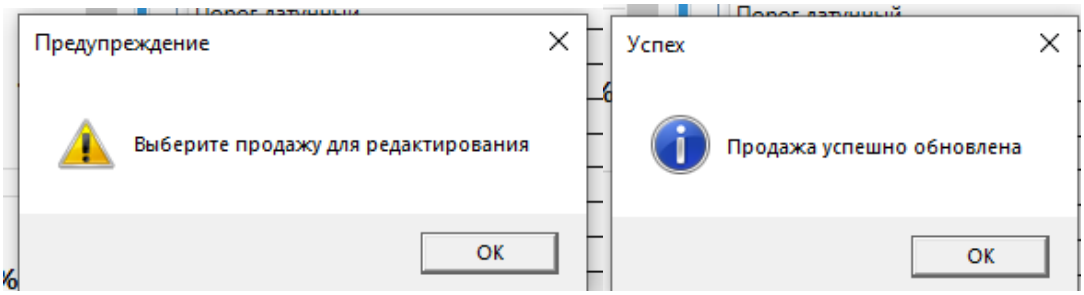

Рисунок 12 – Уведомления

Метод `CalculateTotalAmount` реализует автоматический расчет суммы продажи. При изменении выбранного продукта или количества метод получает цену выбранного продукта из загруженного списка и вычисляет произведение цены на количество. Результат присваивается свойству `TotalAmount`, которое автоматически обновляет интерфейс благодаря реализации `INotifyPropertyChanged`:

```
private void CalculateTotalAmount()
{
    if (ProductComboBox.SelectedValue is int productId && Quantity > 0)
    {
        var product = _products?.FirstOrDefault(p => p.Id == productId);
        if (product != null)
        {
            TotalAmount = product.MinPartnerPrice * Quantity;
        }
    }
    else
    {
        TotalAmount = 0;
    }
}
```

Метод сохранения `SaveButton_Click` реализует комплексную логику: валидацию входных данных, формирование объекта продажи, выбор между операциями добавления и обновления, а также управление результатом диалога. Валидация проверяет выбор продукта, положительность количества и наличие даты продажи. При нарушении правил ввода пользователю выводится соответствующее предупреждение с пояснением причины. При успешном сохранении свойству `DialogResult` присваивается `true`, после чего окно закрывается, а в интерфейсе отображается сообщение об успешном завершении операции.

Метод `DeleteButton_Click` отображает диалог подтверждения с вопросом "Вы уверены, что хотите удалить эту продажу?" и при положительном ответе выполняет асинхронное удаление продажи через репозиторий. В случае успеха устанавливается положительный результат диалога. Обработчик отмены `CancelButton_Click` устанавливает отрицательный результат диалога и закрывает окно, что соответствует стандартному поведению модальных окон.

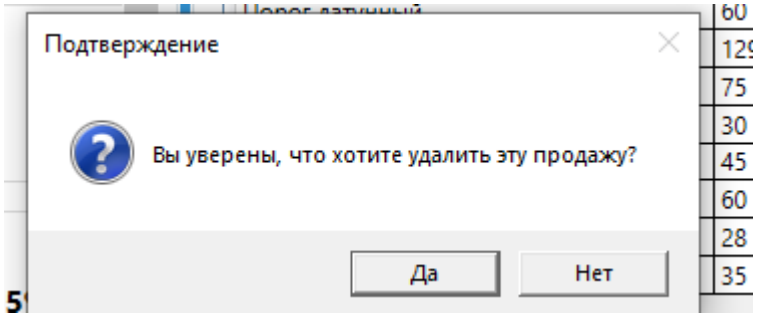

Рисунок 13– Подтверждение об удалении

Все асинхронные методы содержат обработку исключений с выводом информативных сообщений пользователю через `MessageBox`, что обеспечивает обратную связь при возникновении ошибок загрузки, сохранения или удаления данных.

Раздел 2. MarfinaLibrary

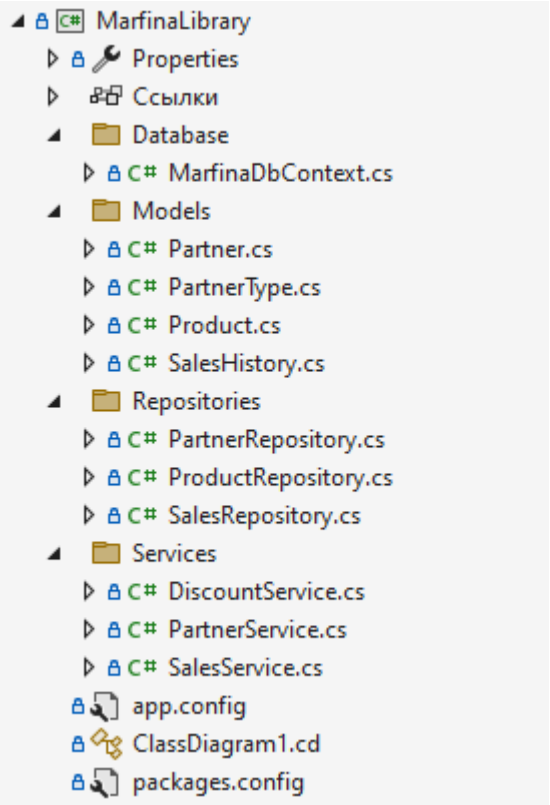

Рисунок 14 – структура библиотеки классов

Проект MarfinaLibrary представляет собой библиотеку классов, реализующую всю серверную логику приложения и обеспечивающую взаимодействие с базой данных. Данный проект является фундаментальным уровнем архитектуры и не содержит элементов пользовательского интерфейса, что позволяет использовать его как независимый компонент в различных клиентских приложениях. Структура проекта организована по многослойному принципу с четким разделением ответственности между компонентами.

Директория Database содержит класс MarfinaDbContext.cs, который является контекстом базы данных Entity Framework Core. Данный класс наследуется от DbContext и определяет наборы данных для работы с таблицами базы, а также содержит конфигурацию подключения к PostgreSQL и настройки индексов и связей между таблицами с использованием Fluent API.

Директория Models включает в себя классы, представляющие сущности предметной области. В ней размещены следующие файлы:

- Partner.cs — модель данных партнера;
- PartnerType.cs — модель типа партнера;
- Product.cs — модель продукции;
- SalesHistory.cs — модель истории продаж.

Каждый класс содержит свойства, соответствующие полям таблиц базы данных, а также навигационные свойства для обеспечения связей между сущностями. Модели используют аннотации данных для явного указания имен таблиц, столбцов и ограничений.

Директория Repositories содержит классы, реализующие паттерн "Репозиторий" для доступа к данным:

- PartnerRepository.cs — предоставляет методы для выполнения операций с данными партнеров, включая получение списка, поиск по идентификатору, добавление, обновление и удаление, а также содержит логику валидации данных;
- SalesRepository.cs — реализует операции с данными о продажах, включая получение истории по партнеру, расчет суммарных показателей и модификацию записей.

Директория Services содержит классы, реализующие бизнес-логику приложения:

- DiscountService.cs — инкапсулирует правила расчета скидок для партнеров на основе объема продаж;
- PartnerService.cs — координирует работу с данными партнеров, объединяя информацию из различных репозиториев и применяя бизнес-правила;
- SalesService.cs — подготавливает данные о продажах для отображения в пользовательском интерфейсе, преобразуя модели данных в модели представления.

В корне проекта размещены вспомогательные файлы. Файл app.config содержит строку подключения к базе данных и другие параметры конфигурации, необходимые для работы библиотеки. Файл packages.config управляет зависимостями проекта, включая пакеты Entity Framework Core и Npgsql для работы с PostgreSQL. Файл ClassDiagram1.cd является файлом диаграммы классов, визуализирующим структуру и взаимосвязи между компонентами библиотеки.

2.1. Диаграмма классов

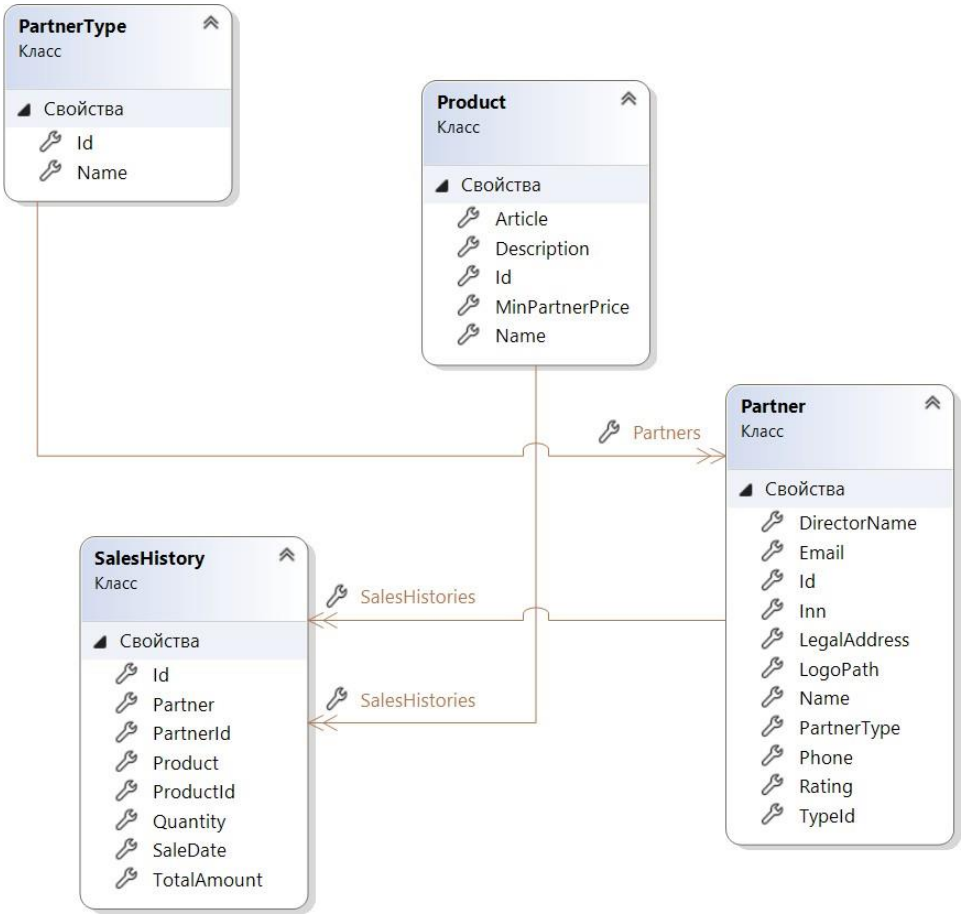

Рисунок 15 – Диаграмма классов

Для объектного представления данных, хранящихся в базе данных, в проекте разработана система классов, полностью отражающая структуру таблиц и связи между ними. Диаграмма классов демонстрирует архитектуру моделей данных, используемых в приложении для работы с информацией о партнерах, продукции и продажах.

Базовым классификатором в системе выступает класс PartnerType, который соответствует таблице типов партнеров. Класс содержит два свойства: идентификатор типа Id и наименование типа Name. Данный класс обеспечивает хранение справочной информации и используется для связывания с основными сущностями.

Класс Product представляет номенклатуру продукции и включает следующие свойства:

- Id — уникальный идентификатор продукта;
- Article — артикул товара;
- Name — наименование продукции;
- Description — развернутое описание товара;
- MinPartnerPrice — минимальная цена для партнеров.

Класс Partner является центральным в модели данных и предназначен для хранения полной информации о партнерах. Он содержит свойства, соответствующие всем полям таблицы partners_marfina, включая идентификатор, ссылку на тип партнера, наименование, контактные данные и рейтинг. На диаграмме класс Partner представлен в виде коллекции Partners, что отражает множество экземпляров данного класса в системе.

Класс SalesHistory фиксирует операции реализации продукции и содержит следующие свойства:

- Id — уникальный идентификатор записи о продаже;
- PartnerId — идентификатор партнера, совершившего покупку;
- ProductId — идентификатор приобретенного продукта;
- Quantity — количество единиц товара;
- SaleDate — дата и время совершения продажи;
- TotalAmount — общая сумма операции;
- Partner — навигационное свойство для доступа к объекту партнера;
- Product — навигационное свойство для доступа к объекту продукта.

Наличие навигационных свойств Partner и Product обеспечивает возможность прямого перехода от записи о продаже к связанным объектам

партнера и продукции без выполнения дополнительных запросов. Это реализует объектно-реляционное отображение связей между таблицами.

Коллекция SalesHistories на диаграмме представляет множество экземпляров класса SalesHistory, что соответствует всем записям о продажах в системе. Аналогично, коллекция Partners представляет множество объектов партнеров.

Связи между классами организованы следующим образом. Класс Partner связан с классом PartnerType отношением "многие к одному", что отражается в наличии свойства-ссылки на тип партнера. Класс SalesHistory связан с классами Partner и Product отношениями "многие к одному", что реализуется через внешние ключи PartnerId и ProductId и соответствующие навигационные свойства.

Таким образом, разработанная диаграмма классов полностью соответствует структуре базы данных и обеспечивает удобное объектное представление информации. Наличие навигационных свойств позволяет эффективно перемещаться по связанным данным, а четкое разделение на отдельные классы соответствует принципам объектно-ориентированного проектирования и способствует поддержанию чистоты архитектуры приложения.

2.2. Database

2.2.1. MarfinaDbContext

Для обеспечения взаимодействия с базой данных PostgreSQL в проекте разработан класс контекста MarfinaDbContext, наследующий от DbContext платформы Entity Framework Core. Данный класс инкапсулирует подключение к базе данных, а также конфигурирует таблицы, индексы и связи между сущностями.

В теле класса определены четыре основных набора данных (DbSet<T>), которые сопоставляются с таблицами в схеме app: PartnerTypesMarfina для типов партнеров, PartnersMarfina для партнеров, ProductsMarfina для

продукции и SalesHistoriesMarfina для истории продаж. Такая структура позволяет выполнять LINQ-запросы напрямую к этим коллекциям, например:

```
public DbSet<Partner> PartnersMarfina { get; set; }
```

Конфигурирование строки подключения реализовано в методе OnConfiguring, где с помощью вызова UseNpgsql задаются параметры доступа к серверу PostgreSQL: хост, имя базы данных, имя пользователя и пароль. Пример настройки:

```
optionsBuilder.UseNpgsql("Host=localhost;Database=marfina_db;Username=app;Password=123456789");
```

Для оптимизации производительности запросов в методе OnModelCreating созданы индексы по внешним ключам и часто используемым полям. Каждый индекс явно именован с помощью метода HasName, что улучшает читаемость и поддержку базы данных. Например, создание индекса по идентификатору типа партнера выглядит следующим образом:

```
modelBuilder.Entity<Partner>()
    .HasIndex(p => p.TypeId)
    .HasName("idx_partners_marfina_type");
```

В этом же методе настроены связи между таблицами с использованием Fluent API. Для отношения между партнером и его типом установлено ограничение на удаление (Restrict), что предотвращает удаление типа, если существуют ссылающиеся на него партнеры. Для связей истории продаж с партнерами и продуктами, напротив, настроено каскадное удаление (Cascade), чтобы при удалении партнера или товара автоматически удалялись все связанные записи о продажах. Пример настройки связи для таблицы продаж:

```
modelBuilder.Entity<SalesHistory>()
    .HasOne(s => s.Partner)
    .WithMany(p => p.SalesHistories)
    .HasForeignKey(s => s.PartnerId)
    .OnDelete(DeleteBehavior.Cascade);
```

Таким образом, реализация MarfinaDbContext полностью покрывает потребности приложения в работе с данными: обеспечивает подключение к СУБД, описывает структуру таблиц, задает индексы для ускорения выборки и

определяет поведение при удалении связанных записей, что гарантирует целостность данных на уровне базы.

2.3. Models

2.3.1. Partner

Для хранения и обработки данных о партнерах в системе разработана модель Partner, представляющая собой сущность, которая сопоставляется с таблицей partners_marfina в схеме app базы данных. Класс использует механизм аннотаций данных (DataAnnotations) для явного указания соответствия полей модели столбцам таблицы, а также для задания ограничений на уровне приложения.

Модель содержит идентификатор записи, помеченный атрибутом [Key], который указывает Entity Framework Core на первичный ключ таблицы. С помощью атрибута [Column] задается точное имя столбца в базе данных, например [Column("id")]. Для обязательных полей, таких как идентификатор типа партнера, наименование и рейтинг, используется атрибут [Required], гарантирующий наличие значения при сохранении. Пример описания ключевого поля и обязательного атрибута:

```
[Key]
[Column("id")]
public int Id { get; set; }

[Required]
[Column("type_id")]
public int TypeId { get; set; }
```

Для строковых полей, где важна длина хранимых данных, применен атрибут [MaxLength], задающий максимальное количество символов. Это позволяет синхронизировать ограничения приложения с ограничениями базы данных. Например, поле телефона ограничено двадцатью символами:

```
[MaxLength(20)]
[Column("phone")]
public string Phone { get; set; }
```

Важной частью модели являются навигационные свойства, обеспечивающие возможность загрузки связанных данных с использованием

механизмов Entity Framework Core. Свойство PartnerType помечено атрибутом [ForeignKey], указывающим, что для связи используется поле TypeId. Это позволяет при выполнении запроса с Include автоматически подгружать информацию о типе партнера. Пример навигационного свойства:

```
[ForeignKey("TypeId")]
public virtual PartnerType PartnerType { get; set; }
```

Кроме того, модель содержит коллекцию SalesHistories, представляющую список всех продаж, связанных с конкретным партнером. Инициализация коллекции пустым списком в объявлении свойства гарантирует отсутствие исключений при добавлении элементов до загрузки данных из базы:

```
public virtual ICollection<SalesHistory> SalesHistories { get; set; } = new List<SalesHistory>();
```

2.3.2. PartnerType

Для классификации партнеров по типам в системе разработана модель PartnerType, представляющая собой справочную сущность, которая сопоставляется с таблицей partner_types_marfina в схеме app базы данных. Класс реализован с использованием аннотаций данных, что позволяет явно задать соответствие между свойствами модели и столбцами таблицы, а также определить ограничения на уровне приложения.

Модель содержит идентификатор типа партнера, помеченный атрибутом [Key], который определяет данное поле как первичный ключ таблицы. С помощью атрибута [Column] задается точное имя столбца в базе данных — id. Для наименования типа партнера используется атрибут [Required], обеспечивающий обязательность заполнения поля, а также атрибут [MaxLength(50)], ограничивающий максимальную длину строки пятьюдесятью символами для согласования с ограничениями базы данных. Пример описания полей модели:

```
[Key]
[Column("id")]
public int Id { get; set; }
```

```
[Required]
[MaxLength(50)]
[Column("name")]
public string Name { get; set; }
```

Важным элементом модели является навигационное свойство Partners, представляющее коллекцию всех партнеров, относящихся к данному типу. Это свойство обеспечивает возможность загрузки связанных данных с использованием механизмов Entity Framework Core, например, при выполнении запроса с включением связанных сущностей. Инициализация коллекции пустым списком при объявлении свойства гарантирует отсутствие исключений при добавлении элементов до фактической загрузки данных из базы:

```
public virtual ICollection<Partner> Partners { get; set; } = new List<Partner>();
```

Реализация модели PartnerType создает справочник типов партнеров, обеспечивает целостность данных на уровне приложения и устанавливает связь "один ко многим" с моделью Partner, что позволяет эффективно выполнять запросы с загрузкой связанных сущностей и поддерживать ссылочную целостность в бизнес-логике приложения.

2.3.3. Product

Для учета товаров и продукции в системе разработана модель Product, представляющая собой сущность, которая сопоставляется с таблицей products_marfina в схеме app базы данных. Класс реализован с использованием аннотаций данных, что позволяет явно задать соответствие между свойствами модели и столбцами таблицы, а также определить ограничения на уровне приложения.

Модель содержит идентификатор продукции, помеченный атрибутом [Key], который определяет данное поле как первичный ключ таблицы. С помощью атрибута [Column] задается точное имя столбца в базе данных — id. Для артикула и наименования продукции используются атрибуты [Required],

обеспечивающие обязательность заполнения полей, а также атрибуты [MaxLength], ограничивающие максимальную длину строк для согласования с ограничениями базы данных. Пример описания ключевых полей:

```
[Key]
[Column("id")]
public int Id { get; set; }

[Required]
[MaxLength(50)]
[Column("article")]
public string Article { get; set; }
```

Поле Description предназначено для хранения описания продукции и не содержит ограничений по длине, что позволяет вносить развернутые характеристики товара. Для хранения минимальной партнерской цены используется тип decimal с явным указанием точности и масштаба через атрибут [Column] с параметром TypeName = "decimal(10,2)", что обеспечивает соответствие формату хранения в базе данных. Пример настройки поля с десятичным числом:

```
[Required]
[Column("min_partner_price", TypeName = "decimal(10,2)")]
public decimal MinPartnerPrice {get; set; }
```

Важным элементом модели является навигационное свойство SalesHistories, представляющее коллекцию всех записей о продажах, связанных с конкретным продуктом. Это свойство обеспечивает возможность загрузки связанных данных с использованием механизмов Entity Framework Core, например, при выполнении запроса с включением истории продаж. Инициализация коллекции пустым списком гарантирует отсутствие исключений при добавлении элементов до фактической загрузки данных из базы:

```
public virtual ICollection<SalesHistory> SalesHistories { get; set; } = new List<SalesHistory>();
```

Реализация модели Product обеспечивает полное соответствие структуре таблицы базы данных, задает необходимые ограничения целостности и

предоставляет механизм для работы со связанными записями о продажах, что является основой для учета товаров и анализа реализации в отчетах системы.

2.3.4. SalesHistory

Для учета и анализа реализации продукции в системе разработана модель SalesHistory, представляющая собой сущность, которая сопоставляется с таблицей sales_history_marfina в схеме app базы данных. Класс реализован с использованием аннотаций данных, что позволяет явно задать соответствие между свойствами модели и столбцами таблицы, а также определить ограничения на уровне приложения.

Модель содержит идентификатор записи о продаже, помеченный атрибутом [Key], который определяет данное поле как первичный ключ таблицы. С помощью атрибута [Column] задается точное имя столбца в базе данных — id. Для всех полей, фиксирующих параметры продажи, используется атрибут [Required], обеспечивающий обязательность заполнения и гарантирующий наличие значения при сохранении. Пример описания ключевого поля и идентификаторов связанных сущностей:

```
[Key]
[Column("id")]
public int Id { get; set; }

[Required]
[Column("partner_id")]
public int PartnerId { get; set; }
```

Для хранения даты и времени совершения продажи применяется тип DateTime с сопоставлением столбцу sale_date. Количество реализованной продукции фиксируется в поле Quantity целочисленного типа, а общая сумма продажи хранится в поле TotalAmount с типом decimal и явным указанием точности и масштаба через атрибут [Column] с параметром TypeName = "decimal(10,2)", что обеспечивает корректное хранение денежных величин в базе данных:

```
[Required]
[Column("total_amount", TypeName = "decimal(10,2)")]
public decimal TotalAmount { get; set; }
```

Важной частью модели являются навигационные свойства Partner и Product, обеспечивающие возможность загрузки связанных данных с использованием механизмов Entity Framework Core. Каждое свойство помечено атрибутом [ForeignKey], указывающим на соответствующее поле внешнего ключа. Это позволяет при выполнении запросов с Include автоматически подгружать информацию о партнере и товаре, связанных с конкретной продажей. Пример навигационного свойства:

```
[ForeignKey("PartnerId")]
public virtual Partner Partner { get; set; }
```

Реализация модели SalesHistory обеспечивает полное соответствие структуре таблицы базы данных, фиксирует все необходимые параметры реализации продукции, задает ограничения целостности и предоставляет удобный механизм для работы со связанными сущностями, что является основой для формирования отчетов о продажах и расчета скидок для партнеров в системе.

2.4. Repositories

2.4.1. PartnerRepository

Для инкапсуляции логики доступа к данным о партнерах в системе разработан класс PartnerRepository, реализующий паттерн "Репозиторий". Класс обеспечивает выполнение операций создания, чтения, обновления и удаления записей о партнерах, а также включает механизмы валидации и корректного освобождения ресурсов.

В конструкторе класса создается экземпляр контекста базы данных MarfinaDbContext, который используется всеми методами для взаимодействия с базой данных. Для обеспечения корректного освобождения неуправляемых ресурсов класс реализует интерфейс IDisposable с стандартным паттерном реализации, что гарантирует вызов метода Dispose для контекста базы данных:

```
public void Dispose()
{    if (!_disposed)
   {        _context?.Dispose();
       _disposed = true;    }}
```

Основные методы репозитория реализованы с использованием асинхронного подхода (async/await), что обеспечивает неблокирующее выполнение операций с базой данных. Метод GetAllPartnersAsync выполняет запрос к таблице партнеров с включением связанных данных о типах партнеров через метод Include, сортируя результаты по наименованию:

```
return await _context.PartnersMarfina
   .Include(p => p.PartnerType)
   .OrderBy(p => p.Name)
   .ToListAsync();
```

Методы добавления и обновления данных включают предварительную валидацию с помощью приватного метода ValidatePartner, который проверяет обязательность наименования, неотрицательность рейтинга и корректность формата электронной почты. В случае нарушения условий валидации выбрасывается исключение с соответствующим сообщением:

```
private void ValidatePartner(Partner partner)
{    if (string.IsNullOrWhiteSpace(partner.Name))
        throw new ArgumentException("Наименование партнера обязательно для заполнения");
```

Для проверки формата электронной почты используется вспомогательный метод IsValidEmail, создающий экземпляр класса MailAddress и сравнивающий результат с исходной строкой. Такой подход обеспечивает надежную валидацию без использования регулярных выражений:

```
private bool IsValidEmail(string email)
{    try
   {   var addr = new System.Net.Mail.MailAddress(email);
       return addr.Address == email;
   }
   catch
   {       return false;    }}
```

Все методы репозитория содержат обработку исключений с последующей генерацией исключений более высокого уровня, включающих информативные сообщения об ошибках. Это позволяет единообразно обрабатывать ошибки доступа к данным на уровне сервисов и

пользовательского интерфейса. Например, при получении списка партнеров ошибка перехватывается и выбрасывается заново с контекстным сообщением:

```
catch (Exception ex)
{
    throw new Exception($"Ошибка при получении списка партнеров: {ex.Message}");
}
```

Реализация PartnerRepository обеспечивает полный набор операций для работы с данными партнеров, включает необходимые проверки целостности, использует современные асинхронные механизмы и гарантирует корректное освобождение ресурсов, что соответствует требованиям к многоуровневой архитектуре приложения.

2.4.2. SalesRepository

Для инкапсуляции логики доступа к данным о реализации продукции в системе разработан класс SalesRepository, реализующий паттерн "Репозиторий". Класс обеспечивает выполнение операций получения информации о продажах, расчета суммарных показателей, а также обновления и удаления записей об операциях реализации.

В конструкторе класса создается экземпляр контекста базы данных MarfinaDbContext, который используется всеми методами для взаимодействия с базой данных. Основные методы репозитория реализованы с использованием асинхронного подхода, что обеспечивает неблокирующее выполнение операций и повышает отзывчивость пользовательского интерфейса при работе с большими объемами данных.

Метод GetSalesHistoryByPartnerAsync выполняет выборку истории продаж для конкретного партнера с включением связанных данных о продукции через метод Include. Результаты сортируются по дате продажи в порядке убывания, что обеспечивает отображение наиболее свежих записей первыми. Пример реализации метода:

```
public async Task<List<SalesHistory>> GetSalesHistoryByPartnerAsync(int partnerId)
{   return await _context.SalesHistoriesMarfina
```

```
        .Include(s => s.Product)
        .Where(s => s.PartnerId == partnerId)
        .OrderByDescending(s => s.SaleDate)
        .ToListAsync();}
```

Для аналитических целей реализован метод GetTotalSalesByPartnerAsync, вычисляющий общую сумму продаж для конкретного партнера с использованием агрегатной функции SumAsync. Это позволяет быстро получать суммарные показатели без загрузки всех записей в память:

```
public async Task<decimal> GetTotalSalesByPartnerAsync(int partnerId)
{
    return await _context.SalesHistoriesMarfina
        .Where(s => s.PartnerId == partnerId)
        .SumAsync(s => s.TotalAmount);
}
```

Метод GetAllSalesWithDetailsAsync предназначен для получения полного списка продаж с включением информации как о партнерах, так и о продукции. Данный метод используется для формирования сводных отчетов и анализа реализации в разрезе всех участников системы:

```
public async Task<List<SalesHistory>> GetAllSalesWithDetailsAsync()
{
    return await _context.SalesHistoriesMarfina
        .Include(s => s.Partner)
        .Include(s => s.Product)
        .OrderByDescending(s => s.SaleDate)
        .ToListAsync();
}
```

Для модификации данных предусмотрены методы UpdateSaleAsync и DeleteSaleAsync. Метод обновления помечает запись как измененную и сохраняет изменения в базе данных, а метод удаления выполняет поиск записи по идентификатору и при ее наличии удаляет из контекста с последующим сохранением:

```
public async Task DeleteSaleAsync(int id)
{
    var sale = await _context.SalesHistoriesMarfina.FindAsync(id);
    if (sale != null)
    {
```

```
        _context.SalesHistoriesMarfina.Remove(sale);
        await _context.SaveChangesAsync();
    }
}
```

Все методы репозитория содержат обработку исключений с последующей генерацией исключений более высокого уровня, включающих информативные сообщения об ошибках. Это обеспечивает единообразный подход к обработке ошибок доступа к данным на уровне сервисов и пользовательского интерфейса. Например, при расчете общей суммы продаж ошибка перехватывается и выбрасывается заново с контекстным сообщением:

```
catch (Exception ex)
{
    throw new Exception($"Ошибка при расчете общей суммы продаж: {ex.Message}");
}
```

Реализация SalesRepository обеспечивает полный набор операций для работы с данными о продажах, включает методы для получения детализированной и агрегированной информации, поддерживает модификацию данных и использует асинхронные механизмы, что соответствует требованиям к многоуровневой архитектуре приложения и обеспечивает эффективное взаимодействие с базой данных.

2.5. Services

2.5.1. DiscountService

Для реализации бизнес-логики определения размера скидки для партнеров в системе разработан класс DiscountService, который инкапсулирует правила расчета процентной скидки на основе общего объема продаж. Сервис содержит методы для вычисления скидки и формирования ее текстового представления.

Основной метод CalculateDiscount принимает в качестве параметра общую сумму продаж партнера и возвращает целочисленное значение скидки в процентах. В начале выполнения метода осуществляется проверка входного параметра на отрицательное значение, при обнаружении которого выбрасывается исключение с соответствующим сообщением. Далее

реализована логика расчета по прогрессивной шкале: при сумме продаж менее 10000 скидка не предоставляется, от 10000 до 50000 устанавливается скидка 5%, от 50000 до 300000 — 10%, а при сумме 300000 и выше — максимальные 15%:

```
public int CalculateDiscount(decimal totalSalesAmount)
{
    if (totalSalesAmount < 0)
        throw new ArgumentException("Сумма продаж не может быть отрицательной");
    if (totalSalesAmount < 10000) return 0;
    else if (totalSalesAmount < 50000) return 5;
    else if (totalSalesAmount < 300000) return 10;
    else return 15;
}
```

Вспомогательный метод GetDiscountDescription предназначен для формирования удобочитаемого текстового описания скидки. Если размер скидки больше нуля, метод возвращает строку с указанием процента, в противном случае возвращается слово "нет", что улучшает восприятие информации при отображении в пользовательском интерфейсе:

```
public string GetDiscountDescription(int discountPercent)
{
    return discountPercent > 0 ? $"{discountPercent}%" : "нет";
}
```

Все методы класса содержат обработку исключений с последующей генерацией исключений более высокого уровня, включающих информативные сообщения об ошибках. Такой подход обеспечивает единообразную обработку ошибок на уровне вызывающих сервисов и представлений. Например, при возникновении исключения в методе расчета оно перехватывается и выбрасывается заново с контекстным сообщением:

```
catch (Exception ex)
{
    throw new Exception($"Ошибка при расчете скидки: {ex.Message}");
}
```

Реализация DiscountService обеспечивает четкое выделение бизнес-логики расчета скидок в отдельный компонент, что соответствует принципам

единственной ответственности и упрощает поддержку и модификацию правил расчета при изменении требований к системе.

2.5.2. PartnerService

В конструкторе сервиса создаются экземпляры необходимых репозиториев и вспомогательных сервисов: PartnerRepository для доступа к данным партнеров, SalesRepository для получения информации о продажах и DiscountService для расчета скидок. Такая композиция обеспечивает четкое разделение ответственности между компонентами:

```
public PartnerService()
{
    _partnerRepository = new PartnerRepository();
    _salesRepository = new SalesRepository();
    _discountService = new DiscountService();
}
```

Метод GetAllPartnersWithDiscountsAsync выполняет получение полного списка партнеров через соответствующий репозиторий. Хотя в текущей реализации метод возвращает только данные партнеров, его архитектура предусматривает возможность расширения для включения рассчитанных скидок непосредственно в возвращаемые объекты:

```
public async Task<List<Partner>> GetAllPartnersWithDiscountsAsync()
{
    var partners = await _partnerRepository.GetAllPartnersAsync();
    return partners;
}
```

Основной функциональностью сервиса является метод GetPartnerWithDiscountAsync, который возвращает составной результат, включающий как данные партнера, так и рассчитанный для него процент скидки. Метод последовательно выполняет получение информации о партнере по идентификатору, проверку его существования, запрос общей суммы продаж из репозитория продаж и расчет скидки с использованием DiscountService. Возврат значений осуществляется с использованием кортежа, что обеспечивает удобную передачу связанных данных:

```
public async Task<(Partner partner, int discountPercent)> GetPartnerWithDiscountAsync(int partnerId)
```

```
{
    var partner = await _partnerRepository.GetPartnerByIdAsync(partnerId);
    if (partner == null)
        throw new Exception("Партнер не найден");

    var totalSales = await _salesRepository.GetTotalSalesByPartnerAsync(partnerId);
    var discountPercent = _discountService.CalculateDiscount(totalSales);

    return (partner, discountPercent);
```

Все методы сервиса содержат обработку исключений с последующей генерацией исключений более высокого уровня, включающих контекстные сообщения об ошибках. Это обеспечивает единообразный подход к обработке ошибок на уровне пользовательского интерфейса и позволяет локализовать источник проблемы при возникновении сбоев:

```
catch (Exception ex)
{
    throw new Exception($"Ошибка в сервисе партнеров: {ex.Message}");
}
```

Реализация PartnerService обеспечивает интеграцию данных из различных источников, применение бизнес-логики расчета скидок и предоставление удобного интерфейса для уровня представления, что соответствует принципам многоуровневой архитектуры и способствует поддержанию чистоты кода в пользовательском интерфейсе.

2.5.3. SalesService

Для подготовки данных о продажах к отображению в пользовательском интерфейсе разработан класс SalesService, реализующий паттерн "Сервис". Класс обеспечивает преобразование данных, полученных из репозитория, в формат, оптимально подходящий для визуализации, и инкапсулирует логику обработки информации о реализации продукции.

В конструкторе сервиса создается экземпляр репозитория продаж SalesRepository, который используется для доступа к данным. Такой подход обеспечивает четкое разделение ответственности между уровнем доступа к данным и уровнем подготовки данных для представления.

Основной метод GetSalesHistoryForPartnerAsync выполняет получение истории продаж для конкретного партнера и преобразует полученные

сущности в модель отображения SalesDisplayModel. В процессе преобразования осуществляется безопасный доступ к связанным данным: при отсутствии информации о товаре подставляется стандартное значение "Неизвестный товар" с использованием оператора объединения с null:

```
public async Task<List<SalesDisplayModel>> GetSalesHistoryForPartnerAsync(int partnerId)
{
    var sales = await _salesRepository.GetSalesHistoryByPartnerAsync(partnerId);
    var displayList = new List<SalesDisplayModel>();

    foreach (var sale in sales)
    {
        displayList.Add(new SalesDisplayModel
        {
            ProductName = sale.Product?.Name ?? "Неизвестный товар",
            Quantity = sale.Quantity,
            SaleDate = sale.SaleDate,
            TotalAmount = sale.TotalAmount
        });
    }

    return displayList;
}
```

Для изоляции модели представления от модели базы данных разработан отдельный класс SalesDisplayModel, который содержит только те поля, которые необходимы для отображения в пользовательском интерфейсе. Данный подход соответствует принципу разделения ответственности и позволяет изменять структуру отображения без влияния на модель данных:

```
public class SalesDisplayModel
{
    public string ProductName { get; set; }
    public int Quantity { get; set; }
    public DateTime SaleDate { get; set; }
    public decimal TotalAmount { get; set; }
}
```

Метод сервиса содержит обработку исключений с последующей генерацией исключения более высокого уровня, включающего контекстное сообщение об ошибке. Это обеспечивает единообразную обработку ошибок на уровне представления и позволяет локализовать источник проблемы:

```
catch (Exception ex)
{
```

```
        throw new Exception($"Ошибка при получении истории продаж: {ex.Message}");
    }
```

Реализация SalesService и SalesDisplayModel обеспечивает четкое разделение между моделями базы данных и моделями представления, выполняет необходимые преобразования данных и предоставляет удобный интерфейс для заполнения элементов пользовательского интерфейса, что соответствует принципам многоуровневой архитектуры и упрощает поддержку и модификацию приложения.

Раздел 3. Схема базы данных

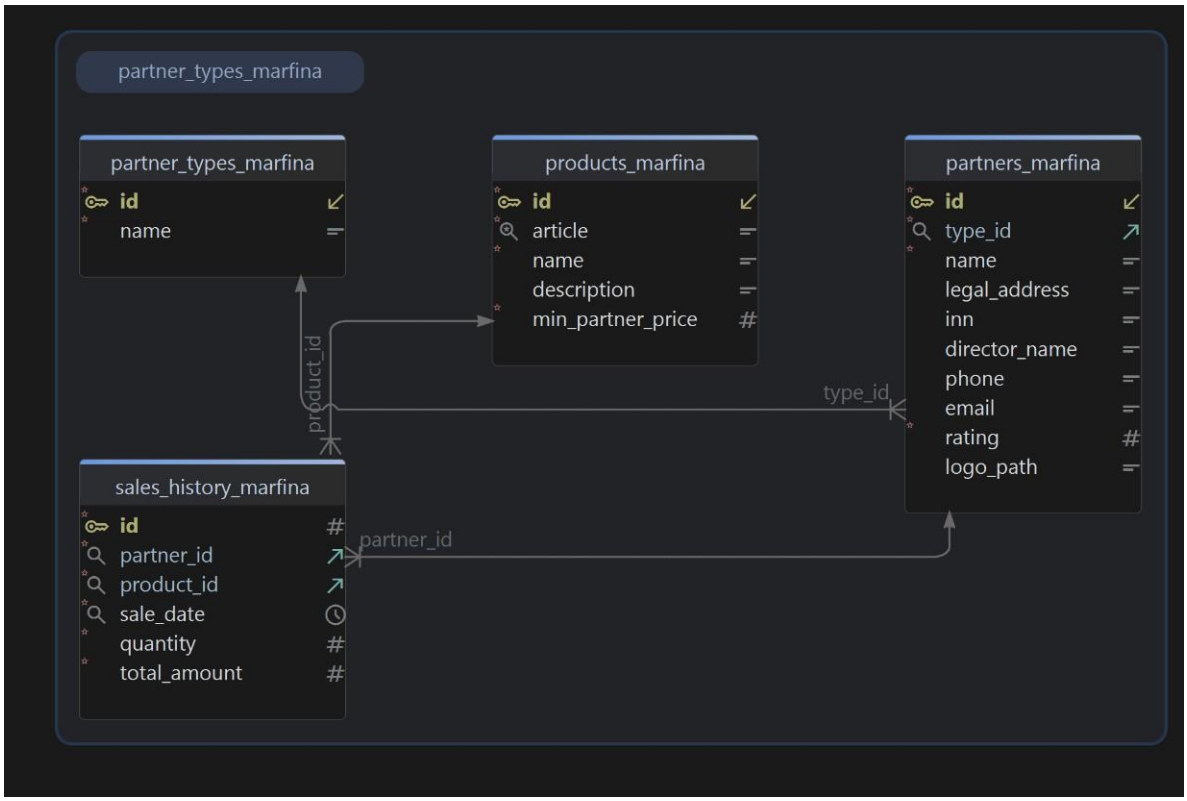

Рисунок 16 – Схема базы данных

Для обеспечения надежного хранения и эффективного доступа к данным в системе разработана реляционная база данных, состоящая из четырех взаимосвязанных таблиц. Все таблицы размещены в схеме app базы данных marfina_db.

Таблица partner_types_marfina является справочником и предназначена для хранения возможных типов партнеров. Она содержит первичный ключ id для уникальной идентификации каждой записи и поле name для наименования типа. Данная таблица обеспечивает классификацию партнеров и позволяет стандартизировать значения, исключая дублирование и ошибки ввода.

Таблица partners_marfina хранит основную информацию о партнерах. Структура таблицы включает следующие поля:

- id — уникальный идентификатор партнера (первичный ключ);
- type_id — внешний ключ, ссылающийся на таблицу типов партнеров;
- name — наименование партнера;
- legal_address — юридический адрес;

- director_name — фамилия, имя и отчество руководителя;

- phone — контактный телефон;

- email — адрес электронной почты;

- rating — рейтинг партнера.

Внешний ключ type_id обеспечивает связь с таблицей partner_types_marfina и гарантирует, что каждому партнеру присвоен существующий тип.

Таблица products_marfina предназначена для хранения номенклатуры продукции. Она содержит следующие поля:

- id — уникальный идентификатор продукта (первичный ключ);

- article — артикул товара;

- name — наименование продукции;

- description — описание товара;

- min_partner_price — минимальная партнерская цена.

Данная таблица обеспечивает единый каталог продукции, реализуемой через партнерскую сеть.

Таблица sales_history_marfina фиксирует все операции реализации продукции и содержит следующие поля:

- id — уникальный идентификатор записи о продаже (первичный ключ);

- partner_id — внешний ключ, ссылающийся на таблицу партнеров;

- product_id — внешний ключ, ссылающийся на таблицу продукции;

- sale_date — дата и время совершения продажи;

- quantity — количество реализованной продукции;

- total_amount — общая сумма продажи.

Внешние ключи partner_id и product_id обеспечивают связь с соответствующими таблицами и позволяют однозначно идентифицировать участников каждой операции реализации. Поле total_amount хранит итоговую стоимость с учетом количества и применяемых скидок.

Связи между таблицами организованы следующим образом. Таблица partners_marfina связана отношением "многие к одному" с таблицей

partner_types_marfina через поле type_id. Таблица sales_history_marfina связана отношением "многие к одному" с таблицей partners_marfina через поле partner_id и с таблицей products_marfina через поле product_id. Такая структура позволяет для каждого партнера хранить множество записей о продажах и для каждого продукта также хранить множество записей о реализации.

Раздел 4. Тестирование

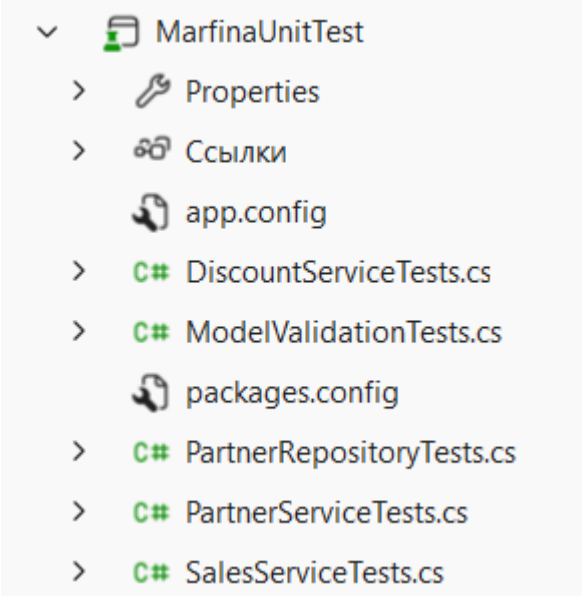

Рисунок 17 – Библиотека тестов

В рамках разработки подсистемы учета партнеров особое внимание уделялось созданию тестовых наборов и сценариев, позволяющих автоматизированно проверить корректность работы ключевых компонентов. Основным инструментом тестирования выбран фреймворк xUnit, который является стандартом де-факто для модульного тестирования в экосистеме .NET и обеспечивает удобные возможности для параметризации тестов, организации фикстур и интеграции со средой разработки Visual Studio.

Разработка unit-тестов для DiscountService. Ключевым компонентом, определяющим корректность бизнес-логики приложения, является сервис расчета скидок DiscountService. Для его тестирования был создан класс DiscountServiceTests, содержащий набор параметризованных тестов, проверяющих все возможные диапазоны сумм продаж. Использование атрибута [Theory] и [InlineData] позволило компактно описать множество тестовых сценариев без дублирования кода. В методе CalculateDiscount_LessThan10000_Returns0 проверяется, что при суммах продаж менее 10000 рублей расчетная скидка составляет 0%.

Разработка unit-тестов для моделей данных. Корректность моделей данных является фундаментом для всей системы, поскольку ошибки в

определении свойств или ограничений могут привести к непредсказуемому поведению на всех уровнях приложения. Для тестирования моделей создан класс ModelValidationTests, содержащий тесты для каждой сущности предметной области. В тесте Partner_WithRequiredFields_IsValid проверяется, что объект партнера может быть создан с минимально необходимым набором свойств (TypeId, Name, Rating), а навигационные коллекции инициализируются пустыми, но не равны null. Тест Partner_WithAllProperties_IsValid проверяет возможность присвоения всех свойств модели, включая необязательные поля, такие как адрес, ИНН, контактные данные и путь к логотипу. Особое значение имеет тест Partner_StringProperties_MaxLength, который проверяет, что строковые свойства модели допускают присвоение значений максимальной длины, указанной в атрибутах [MaxLength], что соответствует ограничениям, наложенным в структуре базы данных. Аналогичные наборы тестов созданы для моделей PartnerType, Product и SalesHistory, что обеспечивает полное покрытие всех сущностей и гарантирует их корректное поведение при использовании в репозиториях и сервисах.

Разработка тестов для сервисов. Для сервисов приложения, выступающих связующим звеном между репозиториями и ViewModel, созданы тестовые классы PartnerServiceTests и SalesServiceTests. В тесте Constructor_CreatesInstance проверяется, что конструктор сервиса успешно создает объект и инициализирует внутренние репозитории. Критически важным является тест GetPartnerWithInvalidId_ThrowsException, который проверяет корректность обработки ситуации, когда запрашивается партнер с заведомо несуществующим идентификатором. Тест ожидает возникновения исключения с помощью Assert.ThrowsAsync<Exception>, что гарантирует, что сервис правильно реагирует на ошибочные ситуации и не возвращает некорректные данные. Хотя набор тестов для сервисов не является исчерпывающим, он покрывает наиболее важные сценарии и демонстрирует подход к тестированию бизнес-логики приложения.

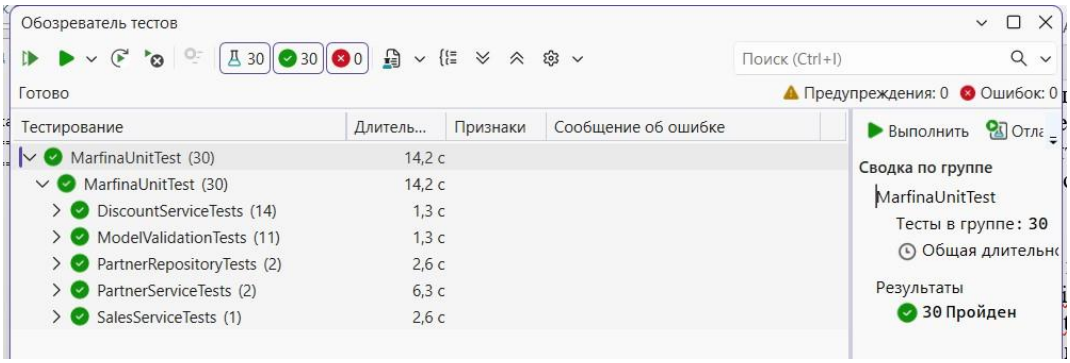

Рисунок 14 - Тестирование

Результатом разработки тестовых наборов и сценариев стал комплект из пяти тестовых классов, содержащих более двадцати отдельных тестов, покрывающих ключевую функциональность приложения. Регулярный запуск этих тестов в процессе разработки позволял своевременно выявлять регрессионные ошибки и гарантировать, что вносимые изменения не нарушают уже работающую функциональность.

Для интеграции компонентов была выбрана нисходящая стратегия (top-down integration), при которой сборка системы начинается с модулей верхнего уровня, управляющих логикой приложения, с постепенным подключением нижележащих компонентов. Такой подход обусловлен архитектурой приложения, построенной по принципу четкой иерархии. В соответствии с планом интеграция выполнялась в четыре основных этапа: интеграция уровня доступа к данным, интеграция сервисного слоя, интеграция ViewModel с сервисами и, наконец, интеграция пользовательского интерфейса с ViewModel. На каждом этапе проводилось тестирование взаимодействия вновь подключаемых модулей с уже интегрированной частью системы, что позволяло локализовать возникающие проблемы и оперативно их устранять.

Интеграция модулей доступа к данным. Первым этапом стала интеграция компонентов, отвечающих за взаимодействие с базой данных. Класс MarfinaDbContext, являющийся центральным элементом Entity Framework Core, был протестирован на корректность настройки подключения к PostgreSQL и правильность маппинга сущностей на таблицы базы данных.

После подтверждения работоспособности контекста к нему были последовательно подключены репозитории PartnerRepository и SalesRepository. На данном этапе проверялась возможность создания экземпляров репозиториев, корректность передачи строки подключения и выполнения простейших запросов к базе данных. Особое внимание уделялось проверке каскадного удаления и ограничений внешних ключей, настроенных в методе OnModelCreating. Для тестирования интеграции использовались контрольные запросы к тестовым данным, загруженным в базу при инициализации, что позволяло убедиться в правильности работы методов GetAllPartnersAsync, GetPartnerByIdAsync и GetTotalSalesByPartnerAsync.

Интеграция сервисного слоя с репозиториями. На втором этапе выполнялась интеграция сервисов DiscountService, PartnerService и SalesService с уже проверенными репозиториями. Сервис DiscountService, не имеющий внешних зависимостей, был протестирован изолированно на предыдущих этапах, поэтому его интеграция заключалась в проверке корректности вызовов из других сервисов. Основная сложность данного этапа заключалась в интеграции PartnerService, который одновременно использует PartnerRepository, SalesRepository и DiscountService. Проверялась способность сервиса получать данные о партнере из репозитория, запрашивать общую сумму продаж из SalesRepository, передавать ее в DiscountService для расчета скидки и возвращать кортеж с партнером и значением скидки.

Интеграция ViewModel с сервисами. Третий этап интеграции являлся ключевым с точки зрения реализации бизнес-логики приложения, поскольку именно на уровне ViewModel происходит взаимодействие пользовательского интерфейса с сервисами и репозиториями. Класс PartnerViewModel интегрировался с PartnerRepository, SalesRepository, DiscountService и SalesService. На данном этапе проверялась корректность инициализации ViewModel, автоматическая загрузка списка партнеров при создании и расчет скидок для каждого партнера с использованием цепочки вызовов: получение партнеров из PartnerRepository, получение суммы продаж из SalesRepository и

вычисление скидки через DiscountService. Особое внимание уделялось асинхронности операций: требовалось убедиться, что загрузка данных не блокирует поток пользовательского интерфейса, а обновление коллекций ObservableCollection происходит в правильном потоке. Для этого в процессе интеграции использовались точки останова и отслеживание идентификатора потока выполнения. Также проверялась корректность реакции на изменение выбранного партнера: при установке свойства SelectedPartner должен автоматически запускаться метод LoadSalesHistoryAsync, обращающийся к SalesService и обновляющий коллекцию SalesHistory.

Интеграция пользовательского интерфейса с ViewModel. Заключительным этапом интеграции стало связывание визуальных компонентов XAML с ViewModel через механизм привязки данных (data binding). В главном окне MainWindow устанавливался экземпляр PartnerViewModel в качестве DataContext, после чего проверялась работа всех привязок. Элемент ListBox с именем PartnersList привязывался к коллекции Partners, а его свойство SelectedItem — к свойству SelectedPartner. Проверялось, что при запуске приложения список автоматически заполняется данными, а при выборе элемента правая панель обновляется. Для DataGrid SalesGrid проверялась привязка к коллекции SalesHistory и корректность отображения данных с использованием форматирования дат и сумм. В окне PartnerEditWindow проверялась передача идентификатора редактируемого партнера из главного окна, загрузка данных в конструкторе и корректное обновление полей формы. Особое внимание уделялось сценариям возврата результата через DialogResult: при успешном сохранении окно должно закрываться с результатом true, инициируя обновление списка в главном окне.

Интеграционное тестирование и устранение дефектов. После завершения всех этапов интеграции проводилось полное интеграционное тестирование, охватывающее все сценарии использования системы в комплексе. Проверялись сценарии добавления нового партнера с последующим отображением в списке, редактирования, существующего с

сохранением изменений, удаления партнера с каскадным удалением связанных записей о продажах и отображения истории продаж для выбранного контрагента. В ходе тестирования были выявлены и устранены отдельные дефекты, связанные с обновлением списка после редактирования (требовался вызов RefreshPartnersAsync) и с обработкой ситуаций, когда у партнера отсутствуют записи о продажах. Все выявленные проблемы были исправлены, и проведено повторное интеграционное тестирование, подтвердившее корректность взаимодействия всех компонентов системы.

Верификация проводилась на протяжении всего жизненного цикла разработки и включала совокупность методов, направленных на контроль качества на различных уровнях детализации. Основными методами верификации, примененными в проекте, являлись инспекции кода, анализ результатов модульного тестирования и проверка соответствия архитектурным решениям. Инспекции кода выполнялись как вручную, так и с использованием автоматизированных средств, таких как анализаторы Roslyn Analyzer и StyleCop, которые контролировали соблюдение стандартов кодирования, включая правила именования идентификаторов, форматирования и организации кода. Результаты модульного тестирования, полученные при выполнении тестовых наборов, описанных в предыдущих разделах, служили количественной мерой корректности реализации отдельных компонентов. Успешное прохождение всех модульных тестов являлось обязательным условием для перехода к этапу интеграции. Дополнительно проводилась верификация архитектурных решений на предмет соответствия выбранному шаблону MVVM и паттерну Repository, что включало проверку наличия четкого разделения ответственности между слоями приложения и отсутствия прямых зависимостей пользовательского интерфейса от логики доступа к данным.

Верификация пользовательского интерфейса. Проверялось наличие всех обязательных элементов: главного меню с командами добавления, редактирования и удаления, двухпанельной компоновки главного окна, списка

партнеров с отображением рассчитанной скидки, таблицы истории продаж с колонками наименования продукции, количества, даты и суммы. Верифицировалось корректное поведение элементов интерфейса: активация пунктов меню редактирования и удаления только при наличии выбранного партнера, автоматическая загрузка истории продаж при выборе партнера, обновление списка после завершения операций добавления и редактирования.

Процесс аттестации программного обеспечения. Аттестация программного модуля проводилась после завершения всех этапов разработки и интеграции и представляла собой формальную процедуру проверки готовности системы к эксплуатации. Целью аттестации являлось подтверждение того, что разработанное приложение полностью реализует функциональные требования, изложенные в техническом задании, и может быть передано заказчику для практического использования. Аттестация включала выполнение полного набора приемочных тестов, охватывающих все сценарии использования системы в условиях, максимально приближенных к реальным.

Проведение аттестационного тестирования. Аттестационное тестирование проводилось в несколько этапов. На первом этапе выполнялось функциональное тестирование в соответствии с разработанными приемочными тестами. Каждый тестовый сценарий выполнялся последовательно. На втором этапе проводилось тестирование надежности, включающее проверку поведения системы при нештатных ситуациях: обрыв соединения с сервером базы данных, попытка выполнения операций с некорректными данными, многократное быстрое выполнение операций добавления и удаления. Проверялось, что приложение корректно обрабатывает такие ситуации, выводя информативные сообщения об ошибках и не допуская аварийного завершения. На третьем этапе выполнялось тестирование удобства использования (usability testing), в ходе которого оценивалась интуитивность интерфейса, понятность сообщений пользователю, соответствие навигации ожиданиям. По результатам

тестирования были внесены незначительные улучшения в формулировки сообщений и расположение элементов управления.

По завершении аттестационного тестирования был проведен анализ его результатов. Все приемочные тесты были пройдены успешно, что подтвердило полную реализацию функциональных требований к системе.